03 Conditions, cycles and libraries in Python

Introduction to Python - RSE course

What did we learn last week?

- **variables** store values
- every variable has a **data type** (string, int...)
- **data structures** store more values (list, tuple, dict...)
- **functions** group instructions into reusable blocks
- functions can have **parameters** (data in) and **return** (data out)
- **local variables** live only inside a specific function
- Python provides **built-in functions** that help us write simpler and shorter code

Conditions

- writing more **complex code**
- allow us to make decisions in our code based on certain criteria
- **if statement** is a *conditional* statement that controls whether a block of code is executed or not
- **else statement** can be used following if to allow us to specify an alternative code block to execute when the if *branch* is not taken
- **elif** (short for “else if”) to provide several alternative choices, each with its own test

Often, you want some combination of things to be true - You can combine relations within a **conditional using and and or**

Conditions - NOT

- **Nested if statements** are not always ideal because they can reduce readability
- But sometimes they help **avoid repeating the same condition many times**
- For example, we may first check **NOT something** and then run other checks inside
- The goal is to keep the code **shorter and more organized**
- Try to add check whether the number is not None in this exercise:

```
# TODO: Write code to check if a number is positive, negative, or zero and print the result
# Hint: Use if, elif, else statements

number = -5

# Your code here
if number > 0:
  print(number, "is postive")
elif number < 0:
  print(number, "is negative")
else:
  print(number, "is zero")
```

Loops

- A *for loop* tells Python to execute some statements once for each value in a list, a character string, or some other collection
- **“for each thing in this group, do these operations”**
- The built-in function range() produces a sequence of numbers.
    - *Not* a list: the numbers are produced on demand to make looping over large ranges more efficient.
    - range(N) is the numbers 0..N-1
- A *while loop* is used when you want to **repeat while** a condition is true

Working with files

- **reading** and **writing** to files
- open the file with function open()
  - **r** - read
  - **w** - write (overwrite)
  - **a** - append
  - recommended using **with** - the file is automatically closed

Libraries - NumPy, math

- well-tested algorithms and tools
- **math** - mathematical functions or constants that are not built-in functions (**sqrt**, **sin**, **log**, **π**, **e**...)
- **NumPy** - library for numerical computations and working with arrays
  - numpy **arrays** - more efficient, support vectorized operations (on entire array at once)
  - functions like **mean, sum** or **standard deviation**
  - generating arrays of **ones** or **zeros**
  - we can handle **NaN values** (missing values)

NumPy is foundation for **data analysis in Python** - follow-up course next week

Key takeaways

- **conditions** allow us make **decisions** in code
- combine conditions with **and** or **or**
- **for** and **while** loops for repeating code
- working with files - **with open(...)**
- NumPy library for numerical computations